

Touchscreen LCD



R. Ferrero, P. Bernardi

Politecnico di Torino

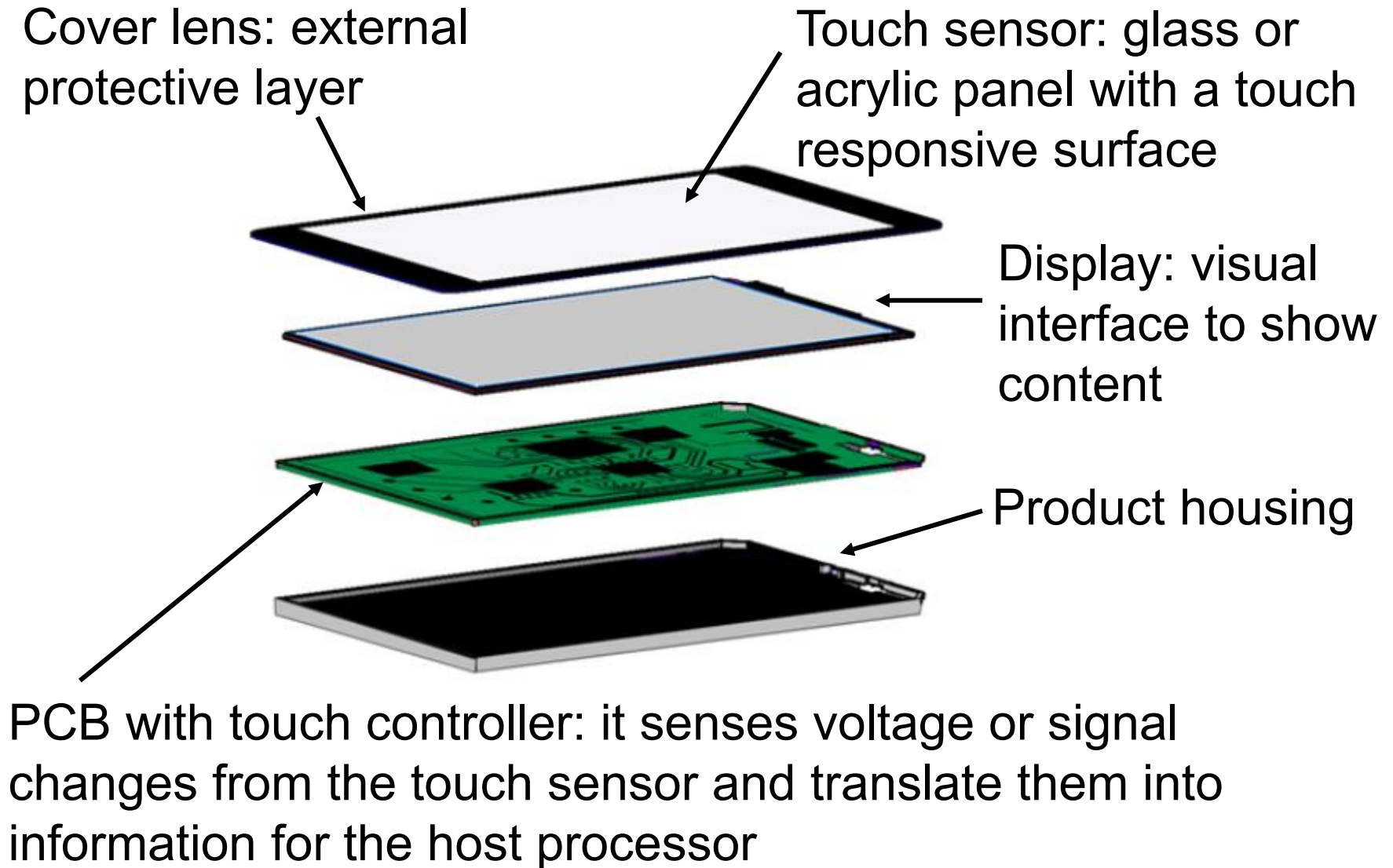
Dipartimento di Automatica e Informatica (DAUIN)

Torino - Italy

This work is licensed under the Creative Commons (CC BY-SA) License. To view a copy of this license, visit <http://creativecommons.org/licenses/by-sa/3.0/>



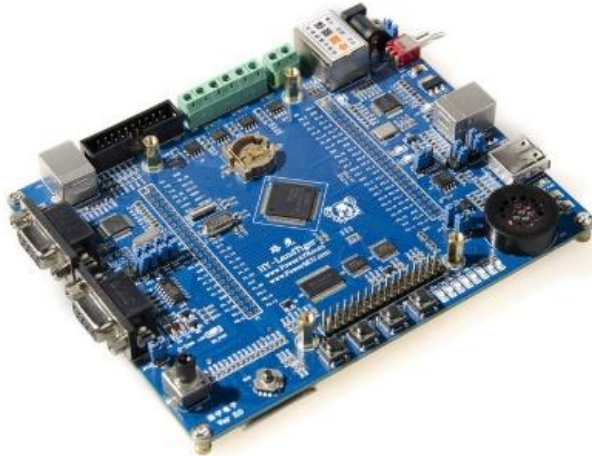
Touch screen components



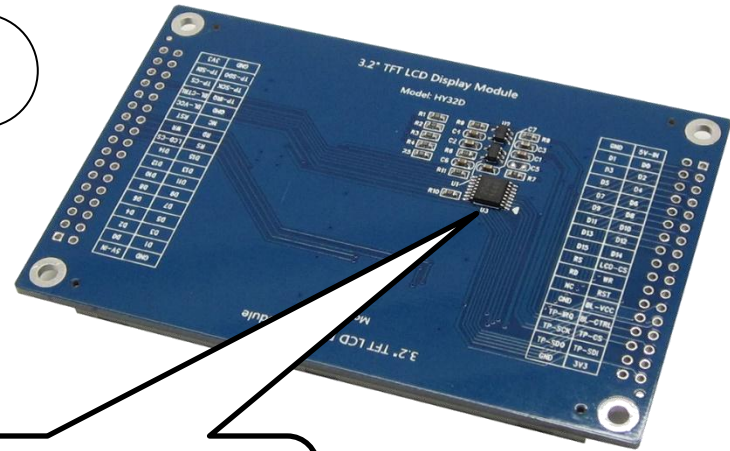
Types of LCD panels

- LCD: liquid-crystal display
 - the liquid crystals have light modulating properties
 - they emit light to produce images
- Thin-Film-Transistor (TFT) LCD
 - a transistor for each pixel improves image quality
- In-Plane Switching (IPS) LCD
 - two glass surfaces align the liquid crystals
- LED-backlit LCD
 - an array of light-emitting diodes (LEDs) illuminate the screen: they are smaller and more efficient.

3.2" touch screen TFT LCD module



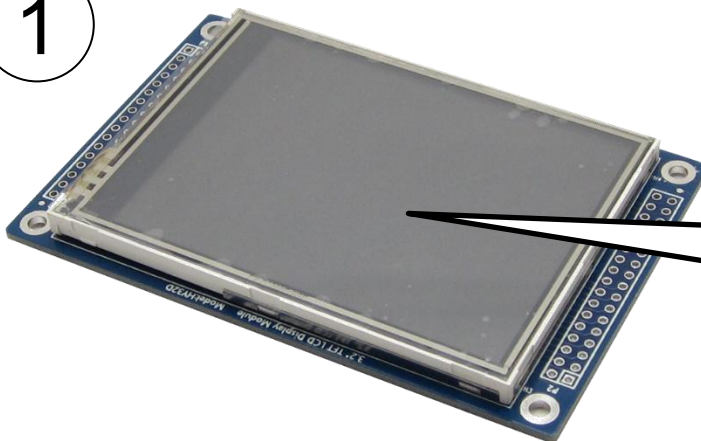
2



Touch screen
controller **ADS7843**



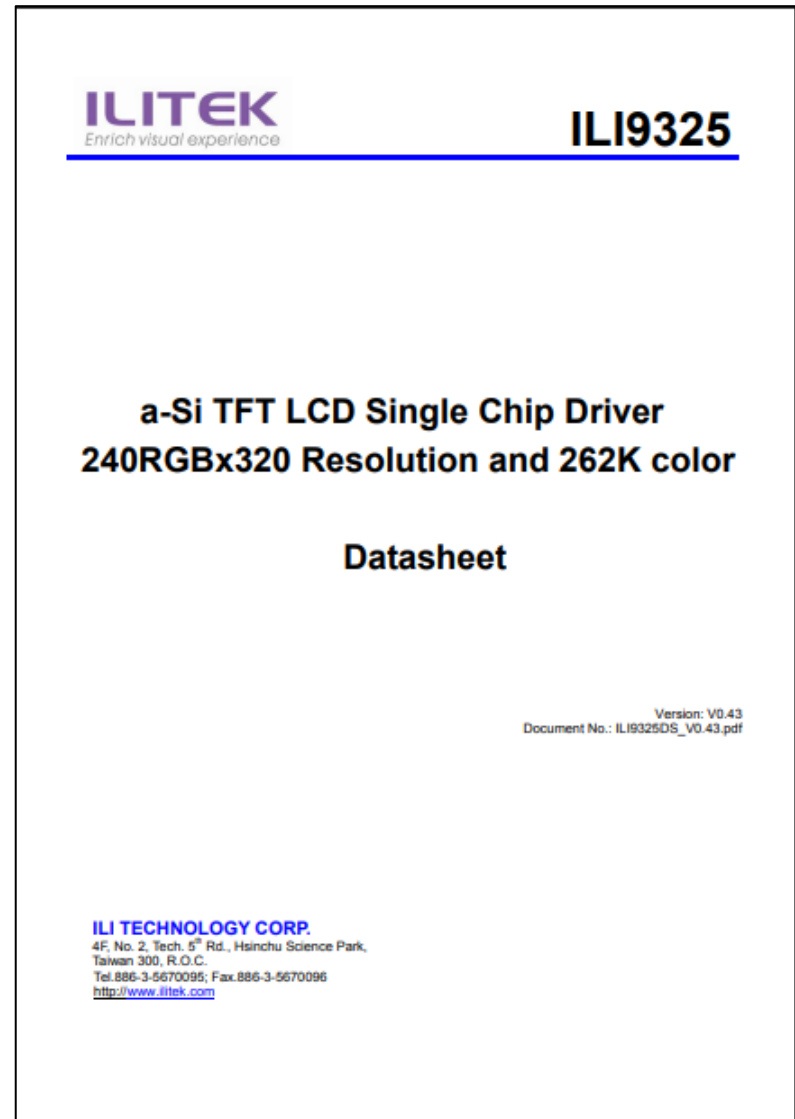
1



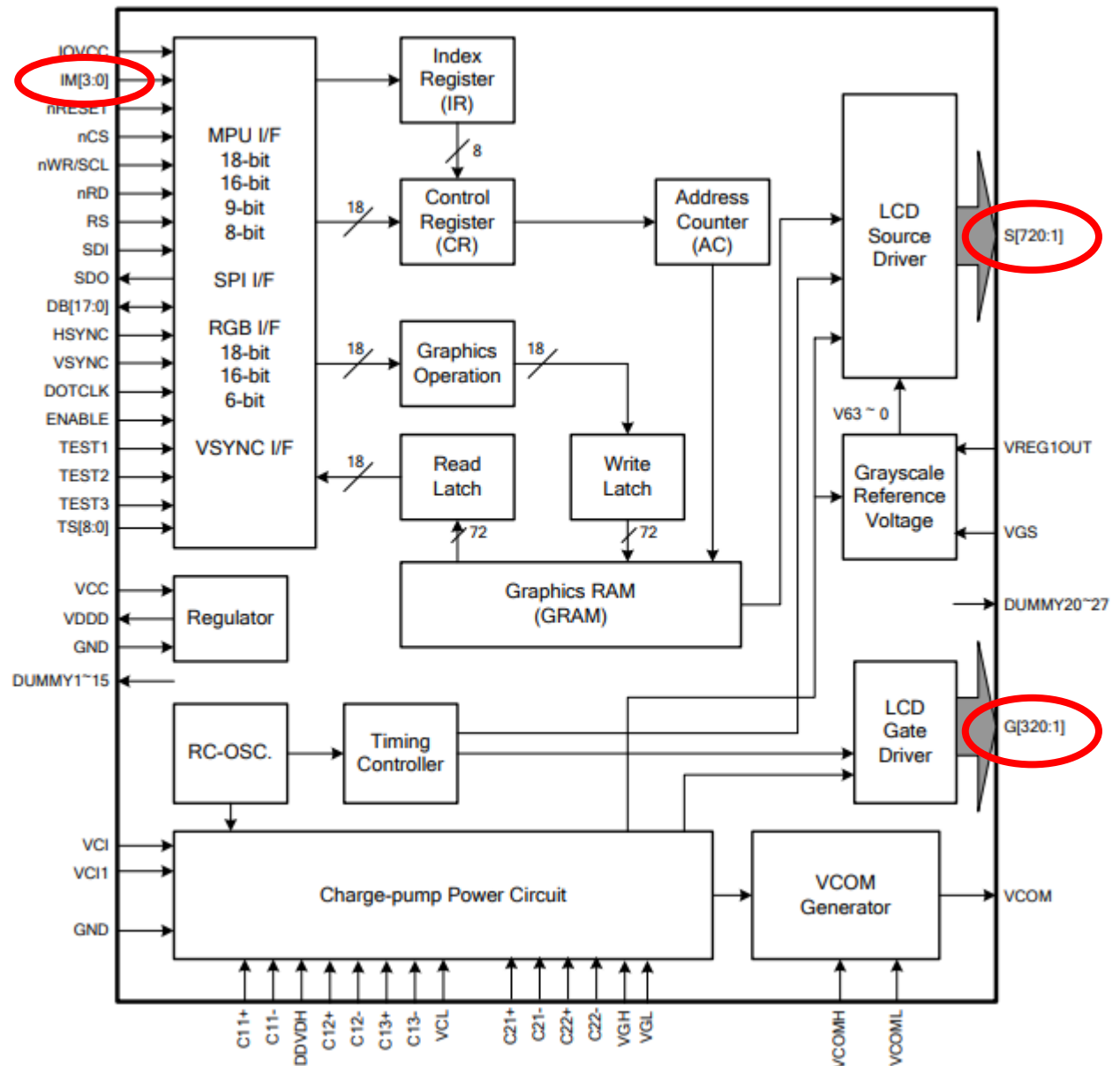
Display controller
ILI9325
(Chip-on-Glass)

1) Display controller ILI9325

- Single chip driver for a TFT LCD display
- 240 RGB x 320-dot resolution
- 262,144 = 2^{18} display color
- 720-channel source driver
- 320-channel gate driver
- 172,800 bytes on-chip RAM
- High-speed RAM burst write function



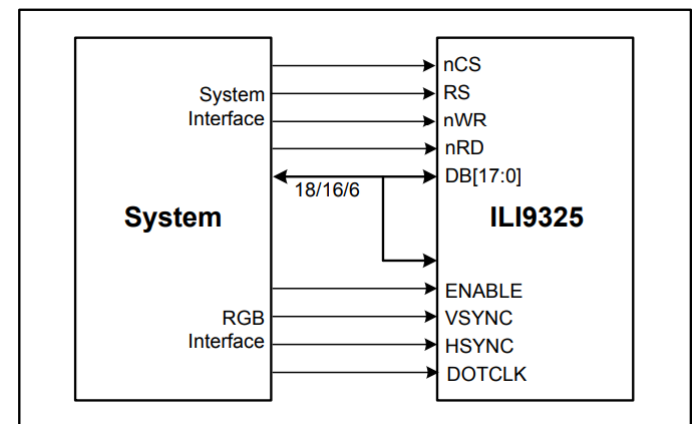
Block diagram



MPU system interface

- Pins IM[3:0] select the MPU interface mode
 - ILI9325 supports i80-system high-speed interfaces to 8-, 9-, 16-, 18-bit parallel ports and serial peripheral interface (SPI)
- The system interface is used for setting registers and GRAM access

IM3	IM2	IM1	IM0	MPU-Interface Mode	DB Pin in use
0	0	0	0	Setting invalid	
0	0	0	1	Setting invalid	
0	0	1	0	i80-system 16-bit interface	DB[17:10], DB[8:1]
0	0	1	1	i80-system 8-bit interface	DB[17:10]
0	1	0	ID	Serial Peripheral Interface (SPI)	SDI, SDO
0	1	1	*	Setting invalid	
1	0	0	0	Setting invalid	
1	0	0	1	Setting invalid	
1	0	1	0	i80-system 18-bit interface	DB[17:0]
1	0	1	1	i80-system 9-bit interface	DB[17:9]
1	1	*	*	Setting invalid	



LCD Driving signals

- Pins S[720:1] provides 3 analog values for every cell in a row (240 pixel)
- Pins G[320:1] select the line to be displayed
 - one line at a time is updated

LCD Driving signals			
S720~S1	O	LCD	Source output voltage signals applied to liquid crystal. To change the shift direction of signal outputs, use the SS bit. SS = "0", the data in the RAM address "h00000" is output from S1. SS = "1", the data in the RAM address "h00000" is output from S720. S1, S4, S7, ... display red (R), S2, S5, S8, ... display green (G), and S3, S6, S9, ... display blue (B) (SS = 0).
G320~G1	O	LCD	Gate line output signals. VGH: the level selecting gate lines VGL: the level not selecting gate lines

Registers

No.	Registers Name	R/W	RS		D15	D14	D13	D12	D11	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0
IR	Index Register	W	0		-	-	-	-	-	-	-	-	ID7	ID6	ID5	ID4	ID3	ID2	ID1	ID0
00h	Driver Code Read	RO	1		1	0	0	1	0	0	1	1	0	0	1	0	0	1	0	1
01h	Driver Output Control 1	W	1		0	0	0	0	0	SM	0	SS	0	0	0	0	0	0	0	0
02h	LCD Driving Control	W	1		0	0	0	0	0	0	BC0	EOR	0	0	0	0	0	0	0	0
03h	Entry Mode	W	1		TRI	DFM	0	BGR	0	0	0	0	ORG	0	I/D1	I/D0	AM	0	0	0
04h	Resize Control	W	1		0	0	0	0	0	0	RCV1	RCV0	0	0	RCH1	RCH0	0	0	RSZ1	RSZ0
07h	Display Control 1	W	1		0	0	PTDE1	PTDE0	0	0	0	BASEE	0	0	GON	DTE	CL	0	D1	D0
08h	Display Control 2	W	1		0	0	0	0	FP3	FP2	FP1	FP0	0	0	0	0	BP3	BP2	BP1	BP0
09h	Display Control 3	W	1		0	0	0	0	0	PTS2	PTS1	PTS0	0	0	PTG1	PTG0	ISC3	ISC2	ISC1	ISC0
0Ah	Display Control 4	W	1		0	0	0	0	0	0	0	0	0	0	0	0	FMARKOE	FMI2	FMI1	FMI0
0Ch	RGB Display Interface Control 1	W	1		0	ENC2	ENC1	ENC0	0	0	0	RM	0	0	DM1	DM0	0	0	RIM1	RIM0
0Dh	Frame Maker Position	W	1		0	0	0	0	0	0	0	FMP8	FMP7	FMP6	FMP5	FMP4	FMP3	FMP2	FMP1	FMP0
0Fh	RGB Display Interface Control 2	W	1		0	0	0	0	0	0	0	0	0	0	0	VSPL	HSPL	0	DPL	EPL
10h	Power Control 1	W	1		0	0	0	SAP	0	BT2	BT1	BT0	APE	AP2	AP1	AP0	0	0	SLP	STB
11h	Power Control 2	W	1		0	0	0	0	0	DC12	DC11	DC10	0	DC02	DC01	DC00	0	VC2	VC1	VC0
12h	Power Control 3	W	1		0	0	0	0	0	0	0	0	VCIRE	0	0	PON	VRH3	VRH2	VRH1	VRH0
13h	Power Control 4	W	1		0	0	0	VDV4	VDV3	VDV2	VDV1	VDV0	0	0	0	0	0	0	0	0
20h	Horizontal GRAM Address Set	W	1		0	0	0	0	0	0	0	0	AD7	AD6	AD5	AD4	AD3	AD2	AD1	AD0
21h	Vertical GRAM Address Set	W	1		0	0	0	0	0	0	0	AD16	AD15	AD14	AD13	AD12	AD11	AD10	AD9	AD8
22h	Write Data to GRAM	W	1		RAM write data (WD17-0) / read data (RD17-0) bits are transferred via different data bus lines according to the selected interfaces.															
29h	Power Control 7	W	1		0	0	0	0	0	0	0	0	0	0	VCM5	VCM4	VCM3	VCM2	VCM1	VCM0
2Bh	Frame Rate and Color Control	W	1		0	0	0	0	0	0	0	0	0	0	0	FRS[3]	FRS[2]	FRS[1]	FRS[0]	
30h	Gamma Control 1	W	1		0	0	0	0	0	KP1[2]	KP1[1]	KP1[0]	0	0	0	0	0	KP0[2]	KP0[1]	KP0[0]
31h	Gamma Control 2	W	1		0	0	0	0	0	KP3[2]	KP3[1]	KP3[0]	0	0	0	0	0	KP2[2]	KP2[1]	KP2[0]
32h	Gamma Control 3	W	1		0	0	0	0	0	KP5[2]	KP5[1]	KP5[0]	0	0	0	0	0	KP4[2]	KP4[1]	KP4[0]
35h	Gamma Control 4	W	1		0	0	0	0	0	RP1[2]	RP1[1]	RP1[0]	0	0	0	0	0	RP0[2]	RP0[1]	RP0[0]
36h	Gamma Control 5	W	1		0	0	0	VRP1[4]	VRP1[3]	VRP1[2]	VRP1[1]	VRP1[0]	0	0	0	0	VRP0[3]	VRP0[2]	VRP0[1]	VRP0[0]
37h	Gamma Control 6	W	1		0	0	0	0	0	KN1[2]	KN1[1]	KN1[0]	0	0	0	0	0	KN0[2]	KN0[1]	KN0[0]
38h	Gamma Control 7	W	1		0	0	0	0	0	KN3[2]	KN3[1]	KN3[0]	0	0	0	0	0	KN2[2]	KN2[1]	KN2[0]
39h	Gamma Control 8	W	1		0	0	0	0	0	KN5[2]	KN5[1]	KN5[0]	0	0	0	0	0	KN4[2]	KN4[1]	KN4[0]
3Ch	Gamma Control 9	W	1		0	0	0	0	0	RN1[2]	RN1[1]	RN1[0]	0	0	0	0	0	RN0[2]	RN0[1]	RN0[0]
3Dh	Gamma Control 10	W	1		0	0	0	VRN1[4]	VRN1[3]	VRN1[2]	VRN1[1]	VRN1[0]	0	0	0	0	VRN0[3]	VRN0[2]	VRN0[1]	VRN0[0]
50h	Horizontal Address Start	W	1		0	0	0	0	0	0	0	0	HSA7	HSA6	HSA5	HSA4	HSA3	HSA2	HSA1	HSA0

Available functions

```
void LCD_Initialization(void);  
void LCD_Clear(uint16_t Color);  
uint16_t LCD_GetPoint(uint16_t Xpos,uint16_t Ypos);  
void LCD_SetPoint(uint16_t Xpos,uint16_t Ypos,uint16_t point);  
void LCD_DrawLine( uint16_t x0, uint16_t y0, uint16_t x1, uint16_t y1 , uint16_t color );  
void PutChar( uint16_t Xpos, uint16_t Ypos, uint8_t ASCII, uint16_t charColor, uint16_t bkColor );  
void GUI_Text(uint16_t Xpos, uint16_t Ypos, uint8_t *str,uint16_t Color, uint16_t bkColor);
```

LCD_Initialization

```
299 void LCD_Initialization(void)
300 {
301     uint16_t DeviceCode;
302     LCD_Configuration();
303     delay_ms(100);
304     DeviceCode = LCD_ReadReg(0x0000); /* 9325; 9328 */
305
306     if( DeviceCode == 0x9325 || DeviceCode == 0x9328 )
307     {
308         LCD_Code = ILI9325;
309         LCD_WriteReg(0x00e7, 0x0010);
310         LCD_WriteReg(0x0000, 0x0001); /* start internal osc */
311         LCD_WriteReg(0x0001, 0x0100);
312         LCD_WriteReg(0x0002, 0x0700); /* power on sequence */
313         LCD_WriteReg(0x0003, (1<<12) | (1<<5) | (1<<4) | (0<<3) ); /*
314         LCD_WriteReg(0x0004, 0x0000);
315         LCD_WriteReg(0x0008, 0x0207);
316         LCD_WriteReg(0x0009, 0x0000);
317         LCD_WriteReg(0x000a, 0x0000); /* display setting */
318         LCD_WriteReg(0x000c, 0x0001); /* display setting */
319         LCD_WriteReg(0x000d, 0x0000);
320         LCD_WriteReg(0x000f, 0x0000);
321         /* Power On sequence */
322     }
```

On-line resources support more than one LCD controller

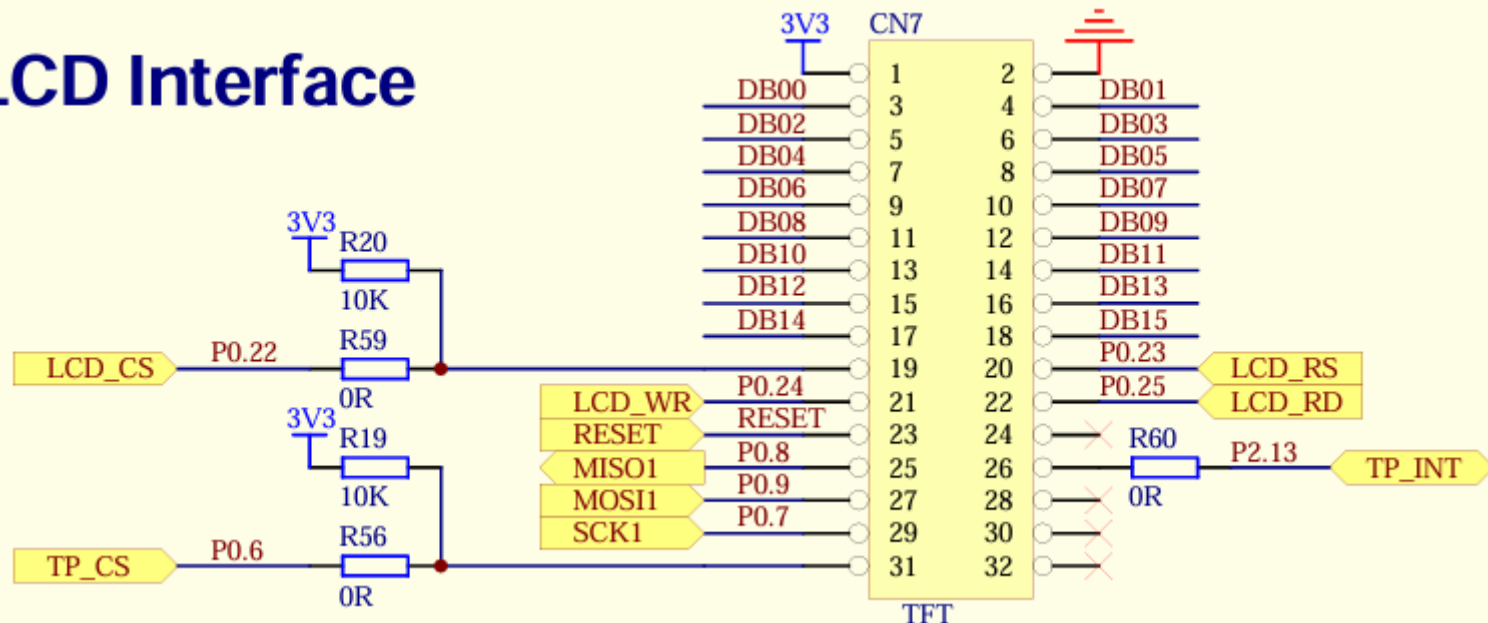
LCD_configuration

```

55 static void LCD_Configuration(void)
56 {
57     /* Configure the LCD Control pins */
58
59     /* EN = P0.19 , LE = P0.20 , DIR = P0.21 , CS = P0.22 , RS = P0.23 , RS = P0.23 */
60     /* RS = P0.23 , WR = P0.24 , RD = P0.25 , DB[0.7] = P2.0...P2.7 , DB[8.15]= P2.0...P2.7 */
61     LPC_GPIO0->FIODIR |= 0x03f80000;
62     LPC_GPIO0->FIOSET = 0x03f80000;
63 }

```

LCD Interface



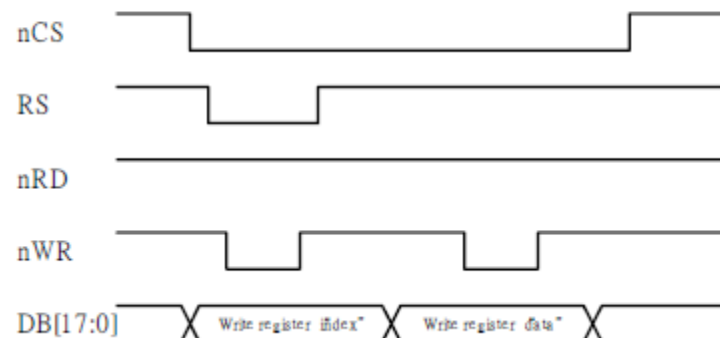
LCD_SetPoint

```
495 void LCD_SetPoint(uint16_t Xpos,uint16_t Ypos,uint16_t point)
496 {
497     if( Xpos >= MAX_X || Ypos >= MAX_Y )
498     {
499         return;
500     }
501     LCD_SetCursor(Xpos,Ypos);
502     LCD_WriteReg(0x0022,point);
503 }
```

```
195 static __attribute__((always_inline)) void LCD_WriteReg(uint16_t LCD_Reg,uint16_t LCD_RegValue)
196 {
197     /* Write 16-bit Index, then Write Reg */
198     LCD_WriteIndex(LCD_Reg);
199     /* Write 16-bit Reg */
200     LCD_WriteData(LCD_RegValue);
201 }
```

i80 18-/16-bit System Bus Interface Timing

(a) Write to register



LCD_WriteIndex and LCD_WriteData

```
129 static __attribute__((always_inline)) void LCD_WriteIndex(uint16_t index)
130 {
131     LCD_CS(0);
132     LCD_RS(0);
133     LCD_RD(1);
134     LCD_Send( index );
135     wait_delay(22);
136     LCD_WR(0);
137     wait_delay(1);
138     LCD_WR(1);
139     LCD_CS(1);
140 }
```

```
142 /*****
```

```
150 static __attribute__((always_inline)) void LCD_WriteData(uint16_t data)
```

```
151 {
152     LCD_CS(0);
153     LCD_RS(1);
154     LCD_Send( data );
155     LCD_WR(0);
156     wait_delay(1);
157     LCD_WR(1);
158     LCD_CS(1);
159 }
160
```

```
30 /* LCD Interface */
```

```
31 #define PIN_EN      (1 << 19)
```

```
32 #define PIN_LE      (1 << 20)
```

```
33 #define PIN_DIR      (1 << 21)
```

```
34 #define PIN_CS      (1 << 22)
```

```
35 #define PIN_RS      (1 << 23)
```

```
36 #define PIN_WR      (1 << 24)
```

```
37 #define PIN_RD      (1 << 25)
```

```
38
39 #define LCD_EN(x)    ((x) ? (LPC_GPIO0->FIOSET = PIN_EN) :
```

```
40 #define LCD_LE(x)    ((x) ? (LPC_GPIO0->FIOSET = PIN_LE) :
```

```
41 #define LCD_DIR(x)   ((x) ? (LPC_GPIO0->FIOSET = PIN_DIR) :
```

```
42 #define LCD_CS(x)    ((x) ? (LPC_GPIO0->FIOSET = PIN_CS) :
```

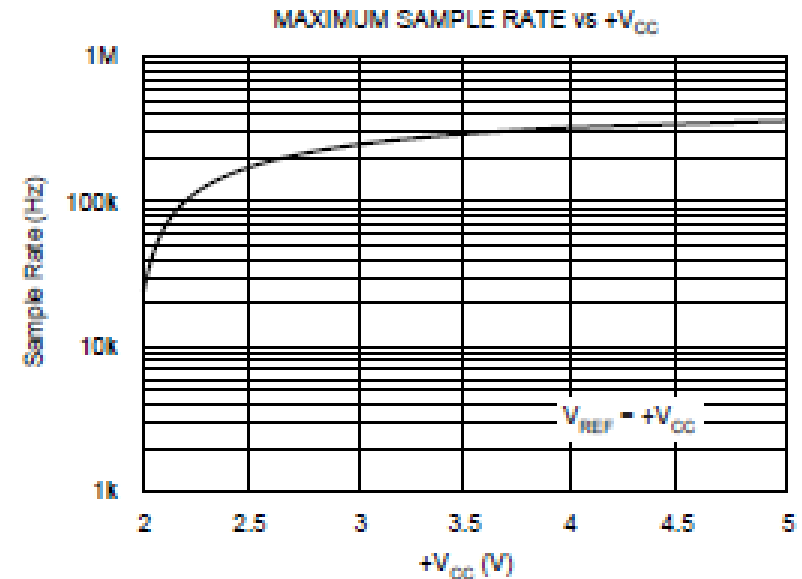
```
43 #define LCD_RS(x)    ((x) ? (LPC_GPIO0->FIOSET = PIN_RS) :
```

```
44 #define LCD_WR(x)    ((x) ? (LPC_GPIO0->FIOSET = PIN_WR) :
```

```
45 #define LCD_RD(x)    ((x) ? (LPC_GPIO0->FIOSET = PIN_RD) :
```

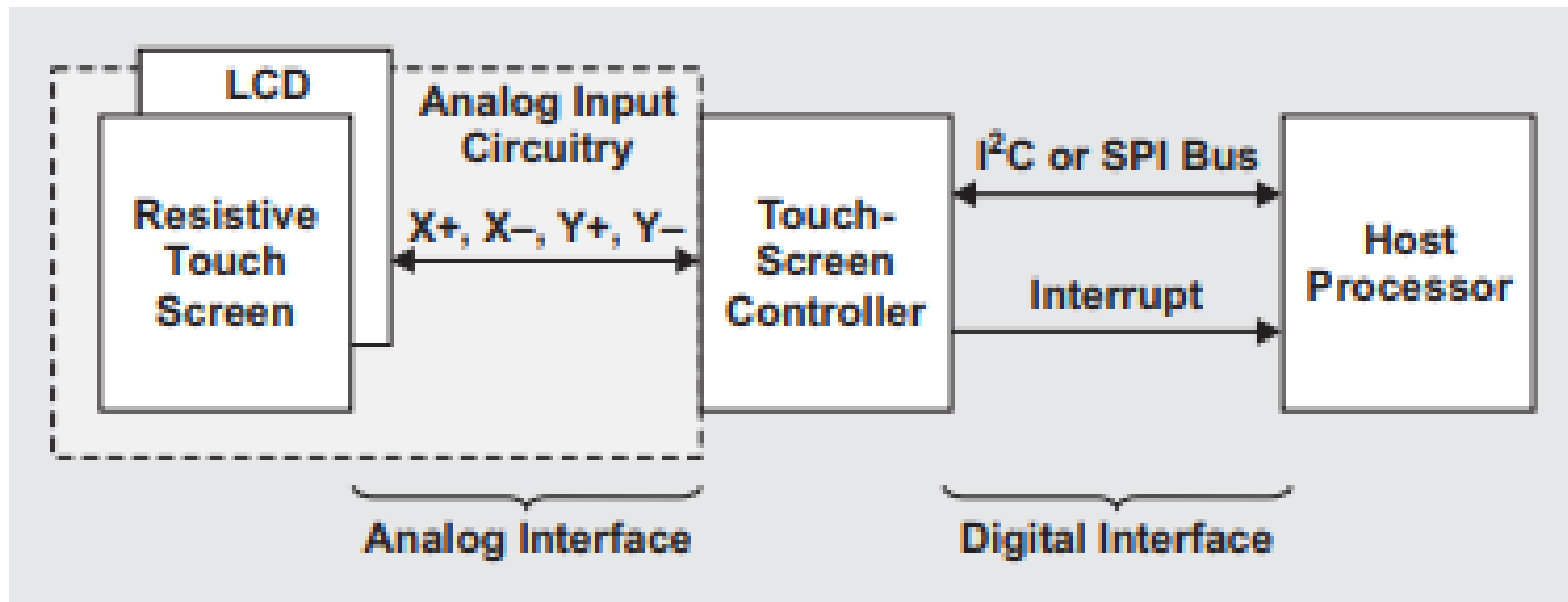
2) Touch screen controller ADS7843

- 4-wire touch screen interface
- ratiometric conversion
- single supply: 2.7v to 5v
- up to 125KHz conversion rate
- serial interface
- programmable 8- or 12-bit resolution
- 2 auxiliary analog inputs
- full power-down control

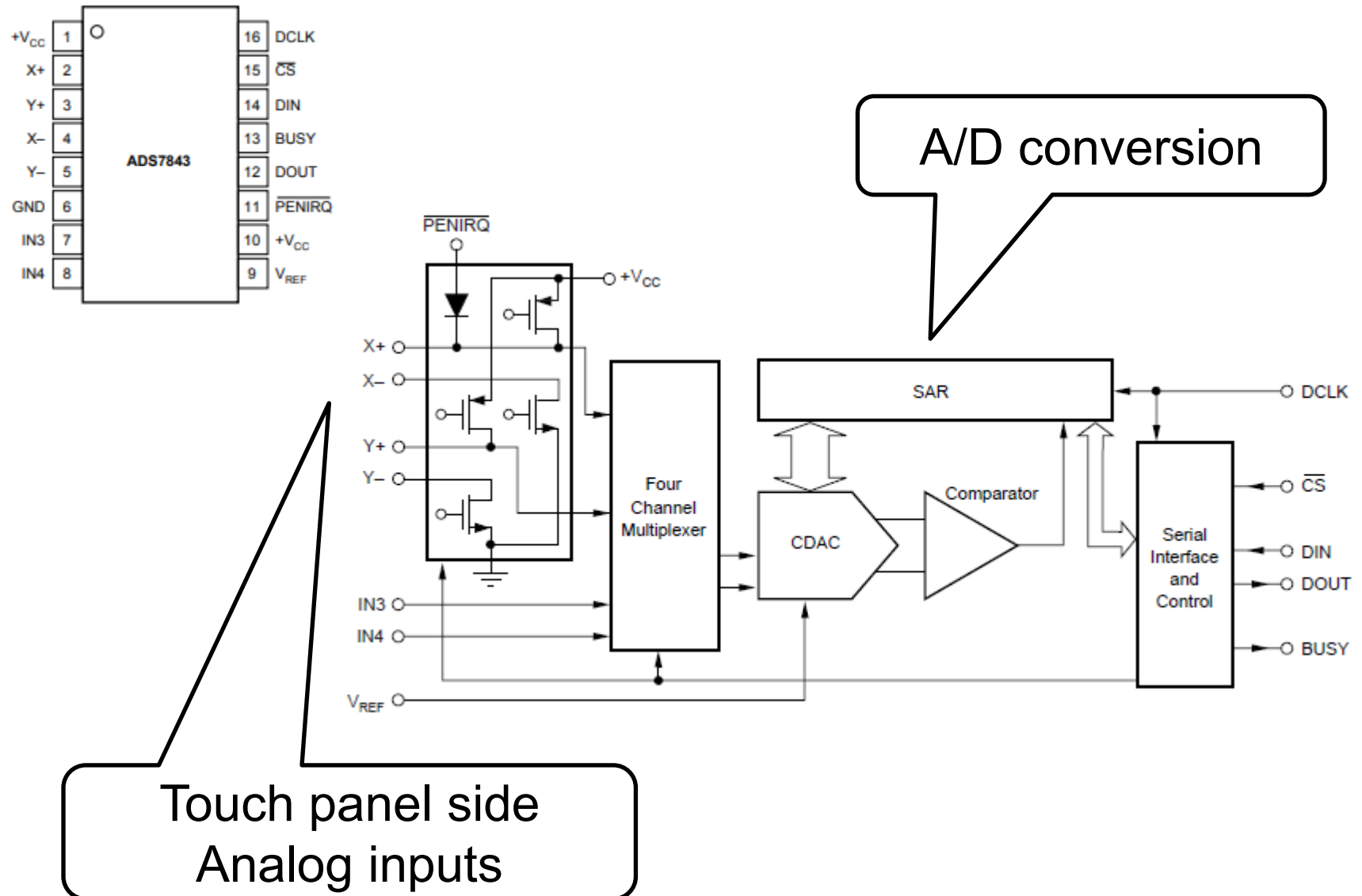


High-level description

- A pressure is applied on the touch sensor
- The ADC converts it into X and Y coordinates
- The touch screen controller sends the data to the host processor via serial interface or I²C.



Block diagram

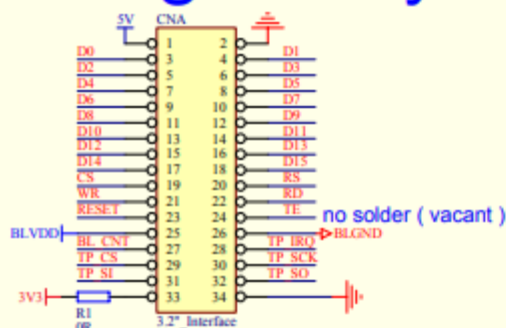


Schematic

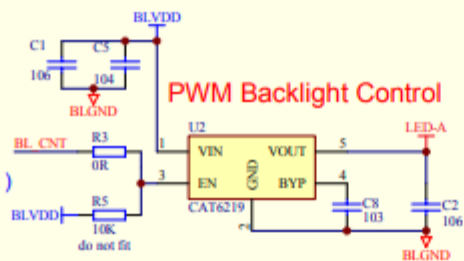
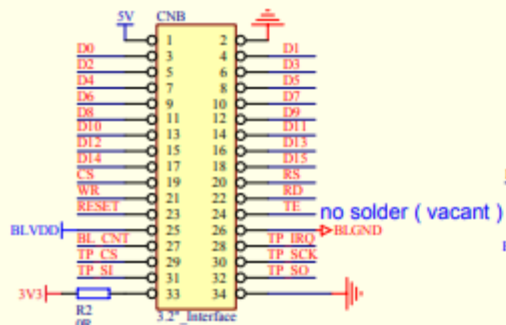
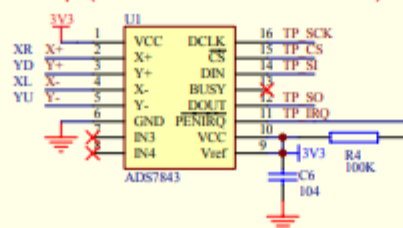
Designed by HAOYU Electronics

www.PowerMCU.com

www.HotMCU.com

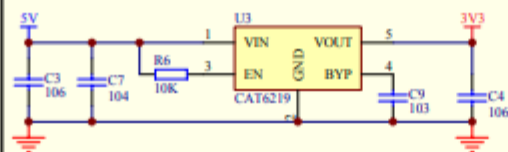


ADS7843 = XPT2046
Touch chip (ADS7843 or XPT2046)

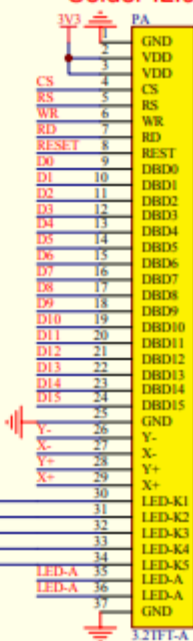


Module description

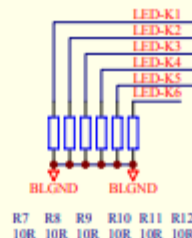
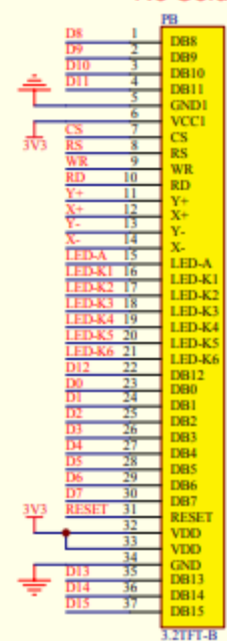
- when 5V power supply, CNA or CNB 1, 2 pin input 5V power, CNA and CNB 33, 34 pin output 3.3V.
- when 3.3V power supply, CNA or CNB 33, 34 pin input 3.3V power, CNA or CNB 1, 2 pin vacant.
- BLVDD BLGND is backlight input power, must input 5V and GND.
- BL_CNT Backlight control pin, PWM wave control adjustable brightness.



Solder ILI9320



No Solder



HAOYU
ELECTRONICS

Title		
Size	Number	Revision
A4		
Date	12-Sep-2010	Sheet of

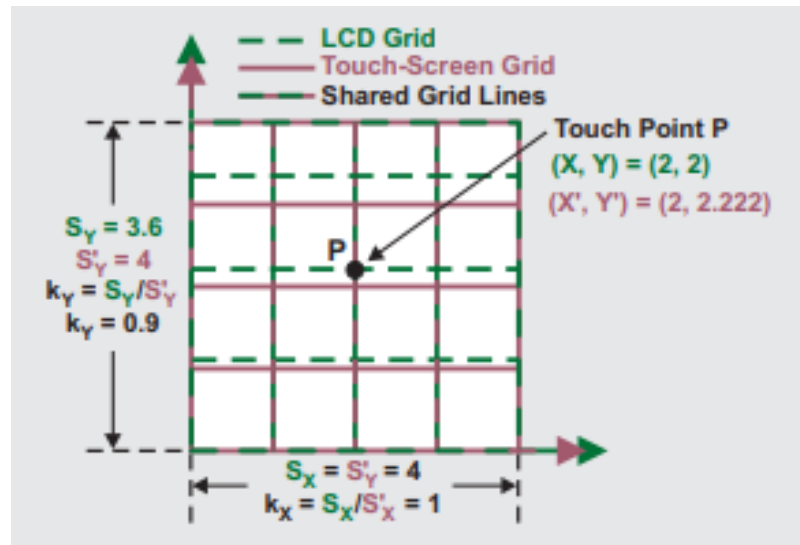
SPI-based communication

```
106 void TP_Init(void)
107 {
108     LPC_GPIO0->FIODIR |= (1<<6); /* P0.6 CS is output */
109     LPC_GPIO2->FIODIR |= (0<<13); /* P2.13 TP_INT is input */
110     TP_CS(1);
111     ADS7843_SPI_Init();
112 }
113
```

```
68 static void ADS7843_SPI_Init(void)
69 {
70     volatile uint32_t dummy;
71
72     /* Initialize and enable the SSP1 Interface module. */
73     LPC_SC->PCONP |= (1 << 10); /* Enable power to SSPI1 block */
74
75     /* P0.7 SCK, P0.8 MISO, P0.9 MOSI are SSP pins. */
76     LPC_PINCON->PINSEL0 &= ~( (3UL<<14) | (3UL<<16) | (3UL<<18) ); /* P0.7,P0.8,P0.9 cleared */
77     LPC_PINCON->PINSEL0 |= (2UL<<14) | (2UL<<16) | (2UL<<18); /* P0.7 SCK1,P0.8 MISO1,P0.9 MOSI1 */
78
79     /* PCLK_SSP1=CCLK */
80     LPC_SC->PCLKSEL0 &= ~(3<<20); /* PCLKSP0 = CCLK/4 (18MHz) */
81     LPC_SC->PCLKSEL0 |= (1<<20); /* PCLKSP0 = CCLK (72MHz) */
82
83     LPC_SSP1->CR0 = 0x0007; /* 8Bit, CPOL=0, CPHA=0 */
84     LPC_SSP1->CR1 = 0x0002; /* SSP1 enable, master */
85
86     LPC17xx_SPI_SetSpeed ( SPI_SPEED_500kHz );
87
88     /* wait for busy gone */
89     while( LPC_SSP1->SR & ( 1 << SSPSR_BSY ) );
90
91     /* drain SPI RX FIFO */
92     while( LPC_SSP1->SR & ( 1 << SSPSR_RNE ) )
93     {
94         dummy = LPC_SSP1->DR;
95     }
96 }
```

Touch-coordinate measurement

- The touch screen controller senses the pressure applied to the touch screen and measures the X and Y coordinates.



TP_GetAdXY

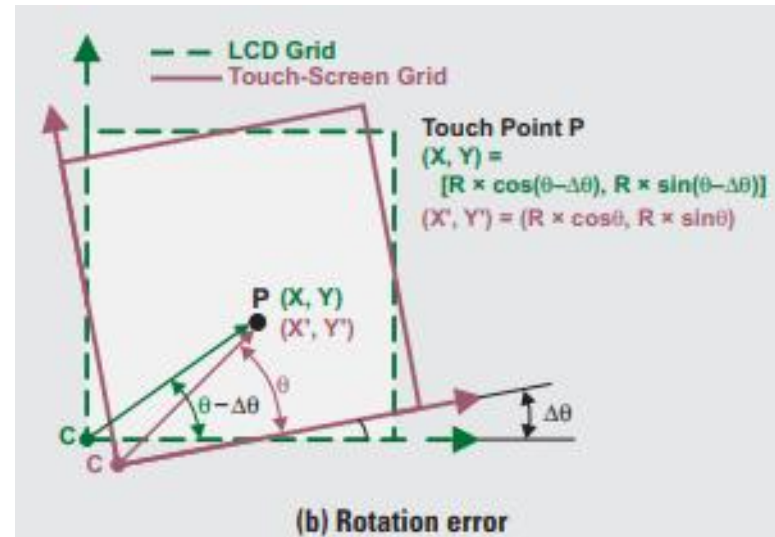
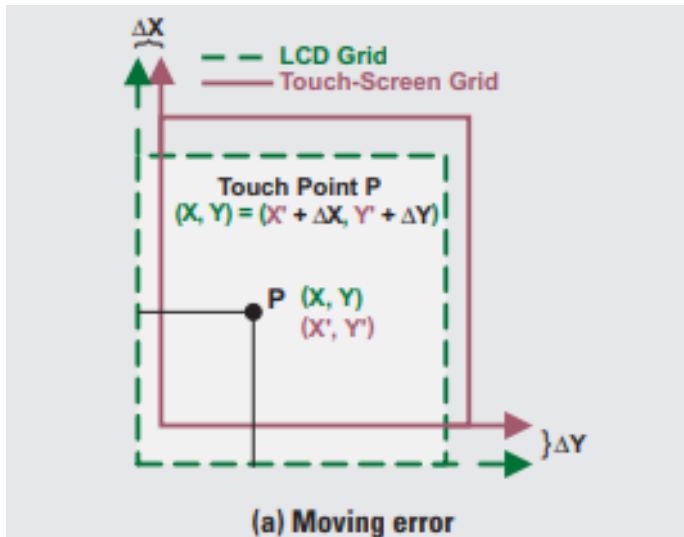
```
231 void TP_GetAdXY(int *x,int *y)
232 {
233     int adx,ady;
234     adx=Read_X();
235     DelayUS(1);
236     ady=Read_Y();
237     *x=adx;
238     *y=ady;
239 }
```

```
190 int Read_X(void)
191 {
192     int i;
193     TP_CS(0);
194     DelayUS(1);
195     WR_CMD(CHX);
196     DelayUS(1);
197     i=RD_AD();
198     TP_CS(1);
199     return i;
200 }
```

```
144 static uint8_t WR_CMD (uint8_t cmd)
145 {
146     uint8_t byte_r;
147
148     while (LPC_SSP1->SR & (1 << SSPSR_BSY) ); /* Wait for transfer to finish */
149     LPC_SSP1->DR = cmd;
150     while (LPC_SSP1->SR & (1 << SSPSR_BSY) ); /* Wait for transfer to finish */
151     while( !( LPC_SSP1->SR & ( 1 << SSPSR_RNE ) ) ); /* Wait until Rx FIFO not empty */
152     byte_r = LPC_SSP1->DR;
153
154     return byte_r; /* Return received value */
155 }
```

Touch-coordinate errors

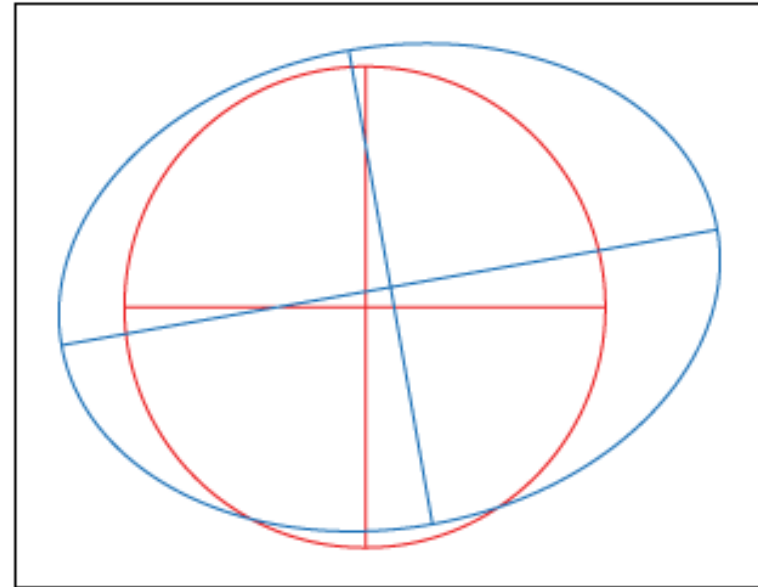
- Several sources of error can affect the accuracy and reliability of this measurement



J. Kostal, "Calibration in Touch Screens System"

Change of the shape on the LCD

- When a finger is traced along the **display** circle (red line), the **touch-panel** system may output the coordinates of an ellipse (blue line) instead of the circle
- The distortion on the LCD touch-screen display (from a circle to an ellipse) is due to translation, rotation, and scaling
 - the view is exaggerated.



Calibration process

- Intuition suggests that any coordinate point (x, y) in an x - y plane that has undergone a transformation should look like:

$$x_{\text{NEW}} = f1(x_{\text{OLD}}, y_{\text{OLD}}) + \text{constant1}$$

$$y_{\text{NEW}} = f2(x_{\text{OLD}}, y_{\text{OLD}}) + \text{constant2}$$

- x_{NEW} and y_{NEW} are the transformed coordinates;
- x_{OLD} and y_{OLD} are the old coordinates;
- $f1()$ and $f2()$ are functions that transform the old coordinates;
- constants1 and 2 are just constants.

Linear calibration

- If the transformation is linear, then functions $f1()$ and $f2()$ can be replaced by the following equations:

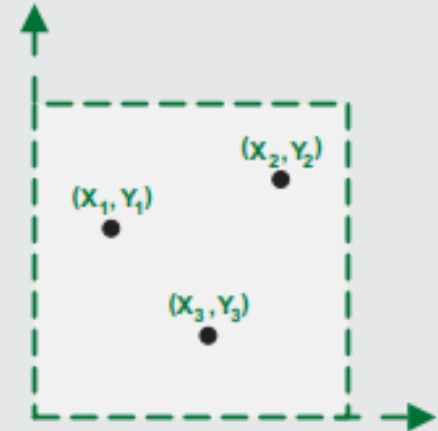
$$x_{NEW} = A x_{OLD} + B y_{OLD} + C$$

$$y_{NEW} = D x_{OLD} + E y_{OLD} + F$$

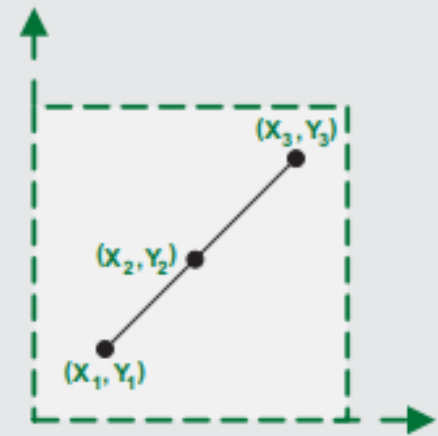
- A, B, C, D, E, and F are constant coefficients
- $f1() = A x_{OLD} + B y_{OLD}$
- $f2() = D x_{OLD} + E y_{OLD}$
- constant1 and constant2 are C and F, respectively

Three-points calibration method

- There are 6 unknowns
 - The values of A, B, C, D, E, and F must be derived
- The 6 required equations are obtained by applying a three-point calibration method to the touch-panel system.



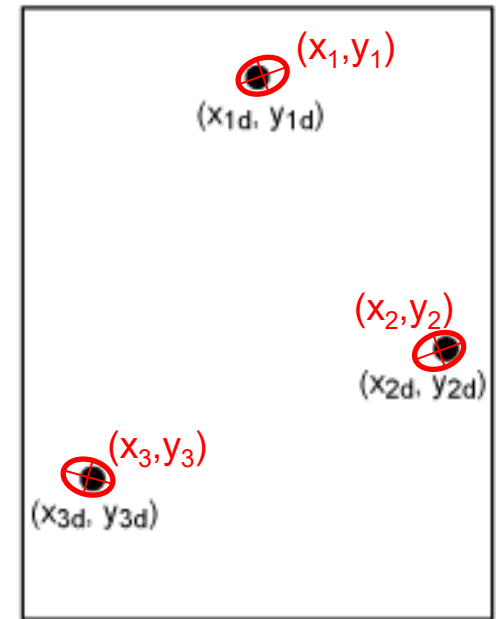
(a) Three independent points



(b) Three dependent points

Three-points calibration method

- 3 pairs of (x, y) coordinates are generated by touching the panel at the 3 pairs of display coordinates: (x_{1d}, y_{1d}) , (x_{2d}, y_{2d}) and (x_{3d}, y_{3d}) .
- Let (x_1, y_1) , (x_2, y_2) , and (x_3, y_3) be the corresponding touch-screen values
- The six unknowns are obtained by solving these equations:



$$x_{1d} = x_1 A + y_1 B + C$$

$$x_{2d} = x_2 A + y_2 B + C$$

$$x_{3d} = x_3 A + y_3 B + C$$

$$y_{1d} = x_1 D + y_1 E + F$$

$$y_{2d} = x_2 D + y_2 E + F$$

$$y_{3d} = x_3 D + y_3 E + F$$

Solution of the equations

$$\begin{pmatrix} X_1 \\ X_2 \\ X_3 \end{pmatrix} = A \times \begin{pmatrix} \alpha_X \\ \beta_X \\ \Delta X \end{pmatrix} \text{ and } \begin{pmatrix} Y_1 \\ Y_2 \\ Y_3 \end{pmatrix} = A \times \begin{pmatrix} \alpha_Y \\ \beta_Y \\ \Delta Y \end{pmatrix},$$

where

$$A = \begin{pmatrix} X'_1 & Y'_1 & 1 \\ X'_2 & Y'_2 & 1 \\ X'_3 & Y'_3 & 1 \end{pmatrix}.$$

Three-point calibration algorithm

Assuming that the dimension of A is 3×3 ,
Equation 8 can be determined from Equation 4,
based on Cramer's rule:

$$\begin{aligned} \alpha_X &= \Delta_{x1}/\Delta, \beta_X = \Delta_{x2}/\Delta, \Delta X = \Delta_{x3}/\Delta, \\ \alpha_Y &= \Delta_{y1}/\Delta, \beta_Y = \Delta_{y2}/\Delta, \text{ and } \Delta Y = \Delta_{y3}/\Delta. \end{aligned} \quad (8)$$

<http://www.ti.com/lit/an/slyt277/slyt277.pdf>

Definitions for Equation 8

$$\Delta = \det(A) = \begin{vmatrix} X'_1 & Y'_1 & 1 \\ X'_2 & Y'_2 & 1 \\ X'_3 & Y'_3 & 1 \end{vmatrix} = (X'_1 - X'_3) \times (Y'_2 - Y'_3) - (X'_2 - X'_3) \times (Y'_1 - Y'_3)$$

$$\Delta_{x1} = \det(A_{x1}) = \begin{vmatrix} X_1 & Y'_1 & 1 \\ X_2 & Y'_2 & 1 \\ X_3 & Y'_3 & 1 \end{vmatrix} = (X_1 - X_3) \times (Y'_2 - Y'_3) - (X_2 - X_3) \times (Y'_1 - Y'_3)$$

$$\Delta_{x2} = \det(A_{x2}) = \begin{vmatrix} X'_1 & X_1 & 1 \\ X'_2 & X_2 & 1 \\ X'_3 & X_3 & 1 \end{vmatrix} = (X'_1 - X'_3) \times (X_2 - X_3) - (X'_2 - X'_3) \times (X_1 - X_3)$$

$$\Delta_{x3} = \det(A_{x3}) = \begin{vmatrix} X'_1 & Y'_1 & X_1 \\ X'_2 & Y'_2 & X_2 \\ X'_3 & Y'_3 & X_3 \end{vmatrix} = X_1 \times (X'_2 Y'_3 - X'_3 Y'_2) - X_2 \times (X'_1 Y'_3 - X'_3 Y'_1) + X_3 \times (X'_1 Y'_2 - X'_2 Y'_1)$$

$$\Delta_{y1} = \det(A_{y1}) = \begin{vmatrix} Y_1 & Y'_1 & 1 \\ Y_2 & Y'_2 & 1 \\ Y_3 & Y'_3 & 1 \end{vmatrix} = (Y_1 - Y_3) \times (Y'_2 - Y'_3) - (Y_2 - Y_3) \times (Y'_1 - Y'_3)$$

$$\Delta_{y2} = \det(A_{y2}) = \begin{vmatrix} X'_1 & Y_1 & 1 \\ X'_2 & Y_2 & 1 \\ X'_3 & Y_3 & 1 \end{vmatrix} = (X'_1 - X'_3) \times (Y_2 - Y_3) - (X'_2 - X'_3) \times (Y_1 - Y_3)$$

$$\Delta_{y3} = \det(A_{y3}) = \begin{vmatrix} X'_1 & Y'_1 & Y_1 \\ X'_2 & Y'_2 & Y_2 \\ X'_3 & Y'_3 & Y_3 \end{vmatrix} = Y_1 \times (X'_2 Y'_3 - X'_3 Y'_2) - Y_2 \times (X'_1 Y'_3 - X'_3 Y'_1) + Y_3 \times (X'_1 Y'_2 - X'_2 Y'_1)$$

Implementation

```
372 uint8_t setCalibrationMatrix( Coordinate * displayPtr,  
373                             Coordinate * screenPtr,  
374                             Matrix * matrixPtr)  
375 {  
376  
377     uint8_t retTHRESHOLD = 0 ;  
378  
379     matrixPtr->Divider = ((screenPtr[0].x - screenPtr[2].x) * (screenPtr[1].y - screenPtr[2].y)) -  
380                         ((screenPtr[1].x - screenPtr[2].x) * (screenPtr[0].y - screenPtr[2].y)) ;  
381     if( matrixPtr->Divider == 0 )  
382     {  
383         retTHRESHOLD = 1;  
384     }  
385     else  
386     {  
387  
388         matrixPtr->An = ((displayPtr[0].x - displayPtr[2].x) * (screenPtr[1].y - screenPtr[2].y)) -  
389                         ((displayPtr[1].x - displayPtr[2].x) * (screenPtr[0].y - screenPtr[2].y)) ;  
390         matrixPtr->Bn = ((screenPtr[0].x - screenPtr[2].x) * (displayPtr[1].x - displayPtr[2].x)) -  
391                         ((displayPtr[0].x - displayPtr[2].x) * (screenPtr[1].x - screenPtr[2].x)) ;  
392         matrixPtr->Cn = (screenPtr[2].x * displayPtr[1].x - screenPtr[1].x * displayPtr[2].x) * screenPtr[0].y +  
393                         (screenPtr[0].x * displayPtr[2].x - screenPtr[2].x * displayPtr[0].x) * screenPtr[1].y +  
394                         (screenPtr[1].x * displayPtr[0].x - screenPtr[0].x * displayPtr[1].x) * screenPtr[2].y ;  
395         matrixPtr->Dn = ((displayPtr[0].y - displayPtr[2].y) * (screenPtr[1].y - screenPtr[2].y)) -  
396                         ((displayPtr[1].y - displayPtr[2].y) * (screenPtr[0].y - screenPtr[2].y)) ;  
397         matrixPtr->En = ((screenPtr[0].x - screenPtr[2].x) * (displayPtr[1].y - displayPtr[2].y)) -  
398                         ((displayPtr[0].y - displayPtr[2].y) * (screenPtr[1].x - screenPtr[2].x)) ;  
399         matrixPtr->Fn = (screenPtr[2].x * displayPtr[1].y - screenPtr[1].x * displayPtr[2].y) * screenPtr[0].y +  
400                         (screenPtr[0].x * displayPtr[2].y - screenPtr[2].x * displayPtr[0].y) * screenPtr[1].y +  
401                         (screenPtr[1].x * displayPtr[0].y - screenPtr[0].x * displayPtr[1].y) * screenPtr[2].y ;  
402     }  
403     return( retTHRESHOLD ) ;  
404 }
```

Calibration with coordinates of 3-points

Data structures

```
30 typedef struct POINT
31 {
32     uint16_t x;
33     uint16_t y;
34 }Coordinate;
35
36
37 typedef struct Matrix
38 {
39     long double An,
40                 Bn,
41                 Cn,
42                 Dn,
43                 En,
44                 Fn,
45                 Divider ;
46 } Matrix ;
47
48 /* Private variables -----
49 extern Coordinate ScreenSample[3];
50 extern Coordinate DisplaySample[3];
51 extern Matrix      matrix ;
52 extern Coordinate display ;
53
```

Adjustment of point position

```
413 uint8_t getDisplayPoint(Coordinate * displayPtr,  
414                        Coordinate * screenPtr,  
415                        Matrix * matrixPtr )  
416 {  
417     uint8_t retTHRESHOLD = 0 ;  
418  
419     if( matrixPtr->Divider != 0 )  
420     {  
421         /* XD = AX+BY+C */  
422         displayPtr->x = ( (matrixPtr->An * screenPtr->x) +  
423                         (matrixPtr->Bn * screenPtr->y) +  
424                         matrixPtr->Cn  
425                         ) / matrixPtr->Divider ;  
426         /* YD = DX+EY+F */  
427         displayPtr->y = ( (matrixPtr->Dn * screenPtr->x) +  
428                         (matrixPtr->En * screenPtr->y) +  
429                         matrixPtr->Fn  
430                         ) / matrixPtr->Divider ;  
431     }  
432     else  
433     {  
434         retTHRESHOLD = 1;  
435     }  
436     return(retTHRESHOLD);  
437 }  
438
```

This function
uses the
calibration
values.

Using touch screen point with timer

- 125 KHz conversion rate corresponds to the period $(125 \text{ KHz})^{-1} = 8 \mu\text{s}$

```
init_timer(0, 0xC8 );           /* 8us * 25MHz = 200 ~= 0xC8 */
```

```
26 void TIMERO_IRQHandler (void)
27 {
28
29     if(getDisplayPoint(&display, Read_Ads7846(), &matrix )){
30         /* Your action here - using display.x and display.y */
31     }
32     else{
33         //do nothing if touch returns values out of bounds
34     }
35     LPC_TIM0->IR = 1;      /* clear interrupt flag */
36     return;
37 }
```